

RACTF 2020 Crypto - Its as easy as access=0000

deuterium — 2020-06-09

Canonical HTML: <https://deut-erium.github.io/WriteUps/2020/ractf/crypto/Its'as'easy'as'access0000/2020-06-09-RACTF-2020-Its-as-easy-as-access-0000.html>

Abstract

CTF writeup: Promote yourself by XOR in RACTF 2020's It's as easy as access=0000: flip AES-CBC IV bits to edit a guest token into admin credentials.

It's as easy as access=0000

The challenge description reads

Challenge instance ready at 95.216.233.106:57735.

We found a strange service, it looks like you can generate an access token for the network service, but you shouldn't be able to read the flag... We think.

And we are provided with server file `access.py`, whose contents read

```
m Crypto.Util.Padding import pad, unpad
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
from datetime import datetime, timedelta

challenge_description("You can generate an access token for my network service, but
you shouldn't be able to read the flag... I think.")
challenge_name = "It's as easy as access=0000"
FLAG = "ractf{XXX}"
KEY = get_random_bytes(16)

def get_flag(token, iv):
    token = bytes.fromhex(token)
    iv = bytes.fromhex(iv)
    try:
        cipher = AES.new(KEY, AES.MODE_CBC, iv)
        decrypted = cipher.decrypt(token)
        unpadded = unpad(decrypted, 16)
    except ValueError as e:
        return {"error": str(e)}
    if b"access=0000" in unpadded:
        return {"flag": FLAG}
    else:
        return {"error": "not authorized to read flag"}

def generate_token():
    expires_at = (datetime.today() + timedelta(days=1)).strftime("%s")
    token = f"access=9999;expiry={expires_at}".encode()
    iv = get_random_bytes(16)
    padded = pad(token, 16)
```

```

    cipher = AES.new(KEY, AES.MODE_CBC, iv)
    encrypted = cipher.encrypt(padded)
    ciphertext = iv.hex() + encrypted.hex()
    return {"token": ciphertext}

def start_challenge():
    menu = "Would you like to:\n[1] Create a guest token\n[2] Read the flag"
    while True:
        print(menu)
        choice = str(input("Your choice: "))
        while choice != "1" and choice != "2":
            choice = str(input("Please enter a valid choice. Try again: "))
        if choice == "1":
            print(generate_token())
        elif choice == "2":
            token = input("Please enter your admin token: ")
            while not token:
                token = input("Tokens can't be empty. Try again: ")
            iv = input("Please enter your token's initialization vector: ")
            while not iv:
                iv = input("Initialization vectors can't be empty. Try again: ")
            print(get_flag(token, iv))

start_challenge()

```

Without reading through whole bunch of code, we can start analyzing from the last, `start_challenge()` is called, which is an infinite loop while `True`, asks for two choices 1 and 2. Choice 1 would invoke `generate_token` function and generate a token. Choice 2 would invoke `get_flag` function, after asking for `iv` and encrypted token.

Now quickly running through `generate_token` line-by-line

```

def generate_token():
    expires_at = (datetime.today() + timedelta(days=1)).strftime("%s")
    token = f"access=9999;expiry={expires_at}".encode()
    iv = get_random_bytes(16)
    padded = pad(token, 16)
    cipher = AES.new(KEY, AES.MODE_CBC, iv)
    encrypted = cipher.encrypt(padded)
    ciphertext = iv.hex() + encrypted.hex()
    return {"token": ciphertext}

```

- It creates an expire time of one day
- Creates a token string starting with `access=9999` specifying guest by 9999 and expiry time
- Generates random IV
- Pads the token to AES block size i.e. 16 bytes
- creates new AES cipher instance
- encrypt the padded token in encrypted
- generates the ciphertext as concatenation of hex of iv and hex of encrypted padded message
- returns the hex concatenation

Running through `get_flag`, one could read

```
def get_flag(token, iv):
    token = bytes.fromhex(token)
    iv = bytes.fromhex(iv)
    try:
        cipher = AES.new(KEY, AES.MODE_CBC, iv)
        decrypted = cipher.decrypt(token)
        unpadded = unpad(decrypted, 16)
    except ValueError as e:
        return {"error": str(e)}
    if b"access=0000" in unpadded:
        return {"flag": FLAG}
    else:
        return {"error": "not authorized to read flag"}
```

- converts user input token and iv to their bytes representation
- Tries decrypting the token with provided iv and its key
- if `access=0000` appears somewhere in message, it must be an admin and returns the flag
- Says FO otherwise

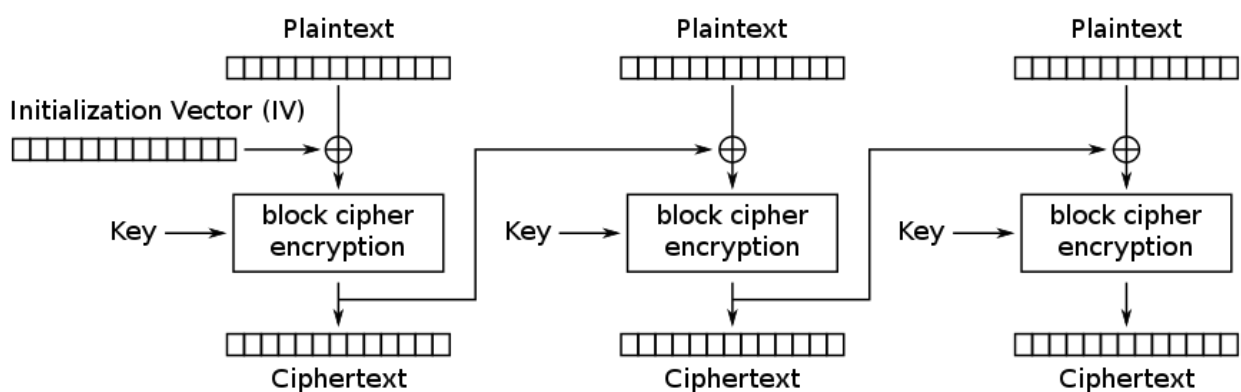
Reading through the above stuff, one could realise what must be done to read the flag 🤔

We need to make the server generate a token for `access=9999` and somehow change it to `access=0000` and fool it into believing that we are admin, and hence the name of the challenge.

This is an instance of all time classic bit flipping attack.

We can modify the IV such that the decryption at server would read `access=0000` at server instead of `access=9999`

Lets see how CBC mode works, just focus on decryption of first block



Cipher Block Chaining (CBC) mode encryption

The output of Block cipher decryption of AES is XORed with the IV to give the plaintext. But hey, It is us who are providing IV, So we must be able to “control” the plaintext with our IV, and hence can make server read whatever we wish.

Lets see how would we do it.

We know $\text{Decrypt}(\text{token}) \oplus \text{IV_server} = \text{access}=9999;<\text{timestamp}>$, where IV_server is the iv provided by default by the server and token is the generated guest token.

To change b'9999' to b'0000', we need to xor something to we $\text{b}'9999' \oplus \text{something} = \text{b}'0000'$, here $\oplus$ is byte-wise xor

$\text{something} = \text{b}'9999' \oplus \text{b}'0000'$

Now lets try xoring something on both sides, (NOTE : I am xoring only the bytes at corresponding position, for the ease of explanation)

$\text{Decrypt}(\text{token}) \oplus \text{IV_server} \oplus \text{something} = \text{access}=9999;<\text{timestamp}> \oplus \text{something}$

$\text{Decrypt}(\text{token}) \oplus \text{IV_server} \oplus \text{something} = \text{access}=9999;<\text{timestamp}> \oplus \text{b}'9999' \oplus \text{b}'0000'$

$\text{Decrypt}(\text{token}) \oplus \text{IV_server} \oplus \text{something} = \text{access}=0000;<\text{timestamp}>$

VOILA! we did it. Lets write a script. One could do that manually, as time is not constrained, but automating stuff is quite a satisfaction.

```
from pwn import remote, xor
import json

HOST, PORT = "95.216.233.106", 58891

REM = remote(HOST, PORT)
print(REM.recvuntil(b'Your choice: ').decode())

REM.sendline(b'1') #get a guest token
data = REM.recvline().replace(b'""',b'""') # badly formatted json
token = json.loads(data)['token']

admin_token = token[32:] # first 32 is iv
iv = bytes.fromhex(token[:32])
iv_xor = b'\x00'*7 + xor(b'9','0')*4 + b'\x00'*5
iv_for_admin = xor(iv_xor, iv).hex()

REM.sendline(b'2')
REM.sendline(admin_token.encode())
REM.sendline(iv_for_admin.encode())
print(REM.recvlines(4))

# ractf{cbc_b17_fl1pp1n6_F7W!}
```

How to make the challenge more difficult

The challenge would be more ~difficult~ interesting if timestamps were modified, and we have to modify the timestamps too to get access 😊.

Nevertheless, awesome work ReallyAwesomeCTF

| Interactive controls are omitted from this PDF. See the canonical HTML version.